

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.Doi Number

Where Does Inference Latency Go? A Controlled Benchmark of In-Process and REST Model Serving for Real-Time Fraud Scoring in Python and on the JVM

Angad Dhiman

Dept. of Computer Science & Engineering, I.K. Gujral Punjab Technical University, Jalandhar, India

Corresponding author: Angad Dhiman (e-mail: angaddhiman10@gmail.com)

The author declares no funding support for this research.

ABSTRACT Fraud scoring in banking back ends typically runs on the Java Virtual Machine (JVM), while models are usually built in Python, so deployments often call a Python REST service for every transaction. Claims that JVM-native inference is orders of magnitude faster rarely separate the costs involved. We present a controlled benchmark of eleven serving configurations that isolates three factors: the inference engine, the host runtime (CPython or JVM), and the transport (in-process call or HTTP). A random forest is trained on the Bank Account Fraud dataset with a month-based temporal split, exported to ONNX with verified score parity (maximum deviation 3.6×10^{-7}), and every arm replays the same 5,000 test transactions in three complete runs. The same ONNX model scores a transaction in a median of 24.6 μ s inside CPython and 33.2 μ s inside the JVM, whereas scikit-learn needs 3.97 ms for the same model, 109 to 174 times longer in every run, a difference due to the engine rather than the language. A correctly configured local REST service adds 0.7–0.9 ms per transaction with a Java client and up to 2.4 ms with a Python client, while a naive development-server deployment costs 45.1 ms, five to eleven times more than a tuned REST service running the same library. A JVM-native Smile forest is fastest (11.9 μ s) but less accurate (ROC-AUC 0.851 versus 0.880). We also document measurement pitfalls on hybrid-core laptops, including OS power throttling that inflated server-side model time roughly threefold, and substantial run-to-run variation of REST latency.

INDEX TERMS Benchmarking, fraud detection, Java Virtual Machine, latency, machine learning model serving, ONNX Runtime, random forest, REST, scikit-learn, Smile

I. INTRODUCTION

Card and account fraud is scored in real time, while a transaction or an account application is still pending, so the scoring step sits on the critical path of authorization [1], [2]. In many banks and payment processors that path is implemented on the Java Virtual Machine (JVM). Fraud models, by contrast, are usually developed in the Python ecosystem. The common way to bridge the two is to wrap the trained model in a Python HTTP service and let the JVM application call it for each transaction, a design that adds a network hop, serialization and a second runtime to every decision and is a well-known source of hidden complexity in production machine learning systems [3].

An alternative is to execute the model inside the JVM, either with a JVM-native library such as Smile or by exporting the Python-trained model to a portable format such

as ONNX and running it with the ONNX Runtime Java API. Reports that JVM-native inference is dramatically faster than a Python REST service are common, but such comparisons frequently change several things at once: the library that evaluates the trees, the language runtime that hosts it, the transport between caller and model, and the configuration of the server. When a single end-to-end ratio is reported, it is impossible to tell which of these factors produced it, and a poorly configured baseline can inflate the ratio several-fold.

This paper asks where the per-transaction latency of such a pipeline actually goes. We benchmark eleven serving configurations (“arms”) that vary one factor at a time around a single trained random forest, and we verify that every arm that claims to run the same model produces the same scores. The study is organized around four research questions:

RQ1: How much of the end-to-end scoring latency is attributable to the inference engine, the host runtime and the transport?

RQ2: With the same trained model and engine, does in-process execution on the JVM reduce latency relative to in-process execution in CPython?

RQ3: How far does a correctly configured Python REST service close the gap to in-process scoring, and how do server and connection settings affect median and tail latency?

RQ4: What accuracy trade-off accompanies a JVM-native library model (Smile) relative to the scikit-learn model?

The contributions of this paper are as follows:

- 1) A controlled, reproducible benchmark design in which engine, runtime and transport are varied independently, the same transactions are replayed in the same order by every arm, and score parity across arms is verified rather than assumed.
- 2) Quantitative evidence, with confidence intervals and effect sizes, that for a random forest the inference engine dominates single-transaction latency, that the host runtime has a small effect when the engine is held fixed, and that a correctly configured local HTTP hop costs on the order of a millisecond, with marked run-to-run variation on client hardware.
- 3) A re-examination of a naive development-server baseline showing that it overstates the cost of REST serving five- to elevenfold.
- 4) An evaluation on the Bank Account Fraud dataset with a month-based temporal split and a fixed false-positive-rate operating point, and a documented set of measurement pitfalls on hybrid-core laptops that can silently distort latency results.

The remainder of the paper is organized as follows. Section II reviews related work. Section III describes the dataset, models, benchmark arms, measurement protocol and statistical analysis. Section IV reports classification and latency results, Section V discusses their implications and threats to validity, and Section VI concludes.

II. RELATED WORK

A. FRAUD DETECTION AND ITS EVALUATION

Statistical fraud detection has long been framed as the problem of finding rare events in high-volume transaction streams [1]. Comparative studies established tree ensembles, and random forests in particular [4], as strong baselines for card fraud [5], and later work combined supervised and unsupervised signals [6]. Dal Pozzolo et al. showed that realistic evaluation must respect the temporal order of transactions and the delayed availability of labels [2]. Class imbalance is a central difficulty, addressed by resampling such as SMOTE [7] or by cost-sensitive weighting [8]. Under strong imbalance, precision-recall curves are more informative than ROC curves [9], and the Matthews correlation coefficient summarizes a confusion matrix more faithfully than accuracy or F1 [10]. The Bank Account Fraud

(BAF) suite [11] provides large, privacy-preserving synthetic datasets derived from real account-opening fraud data, with a month attribute that permits temporal splits and a recommended evaluation at a fixed false-positive rate.

B. MODEL SERVING AND LATENCY

Dedicated prediction-serving systems such as Clipper [12] and TensorFlow Serving [13] showed that batching, caching and the serving architecture strongly influence latency, and that tail latency, not the mean, determines user-visible behaviour in large services [14]. For tree ensembles, several studies have compared inference engines directly. Hummingbird compiles tree models into tensor programs and reports that ONNX-ML scores single records faster than scikit-learn [15], and a broader comparison of decision-forest inference pipelines found that end-to-end latency depends strongly on the platform and on data-transfer overheads [16]. Portable model formats also allow a model to be embedded in the caller's process across languages: ONNX Runtime [17] executes models exported from scikit-learn [18] in several host languages, including Java, and JVM-native libraries such as Smile [19] train and score models directly on the JVM. Libraries such as the Deep Java Library [20] extend this approach to deep models. In CPython, the global interpreter lock limits parallel execution of Python-level code within one process [21], which matters for how Python inference servers are configured.

The study closest to ours is that of Bondaruk et al. [22], who executed the same IsolationForest model through a Python/FastAPI sidecar and through ONNX Runtime embedded in a Spring Boot service and found that embedding reduced mean request latency from 26.1 to 4.8 ms. Their comparison, however, changes the inference engine, the host runtime and the transport at the same time, and the authors note that in-process ONNX Runtime in Python was not evaluated. The present study adds exactly these missing arms. By holding the trained model fixed and adding in-process scikit-learn and ONNX Runtime in CPython, the same ONNX model in the JVM, and REST services with both engines and both client languages, it attributes the latency difference separately to engine, runtime and transport, for a fraud-detection workload with a temporal evaluation protocol.

C. BENCHMARKING METHODOLOGY

Managed runtimes make timing difficult: just-in-time compilation, garbage collection and non-deterministic optimization produce warm-up phases and run-to-run variation. Georges et al. argued that Java performance results must be reported with statistically rigorous methods [23], and Kalibera and Jones showed how to plan repeated experiments so that variation between runs is captured [24]. Latency samples are skewed and serially dependent, which motivates rank-based tests [25], ordinal effect sizes [26] and bootstrap confidence intervals [27], with block resampling for dependent observations [28]. We follow these recommendations and report medians, tail percentiles, confidence intervals and effect sizes rather than a single mean ratio.

III. EXPERIMENTAL DESIGN

A. DATASET AND TEMPORAL SPLIT

We use the Base variant of the BAF suite [11], obtained from Kaggle under the CCBY-NC-SA 4.0 license. It contains 1,000,000 account-opening applications described by 32 columns, of which 11,029 (1.10%) are labelled as fraud. The data are synthetic: they were derived from a real account-opening fraud dataset using differential-privacy noise addition, feature encoding and a CTGAN generative model [11]; results therefore characterize this benchmark distribution and are not claims about any specific institution's transactions. Missing values are encoded in the source as -1 and were left as such, since tree ensembles can split on the sentinel value. The column device_fraud_count is constant and was removed; the month column was used only for splitting. The remaining 29 features include five categorical attributes (payment_type, employment_status, housing_status, source and device_os), which were ordinarily encoded by mapping each level observed in the training months to its index in sorted order; the encoders were fitted on the training months only, and constant columns were identified on the training months as well.

Following the recommendation to respect temporal order [2], months 0–4 form the training set (675,666 rows, 6,740 fraud, 1.00%), month 5 the validation set (119,323 rows, 1,411 fraud, 1.18%) and months 6–7 the test set (205,011 rows, 2,878 fraud, 1.40%). The validation month is used only to choose decision thresholds; the test months are used only for the final evaluation. The rising fraud rate across the three periods is itself a form of drift that a random shuffle would hide.

B. MODELS

The reference model is a scikit-learn random forest [18], [4]. Table I lists its hyperparameters. Class imbalance is handled with a class weight of 90 for fraud, approximately the inverse prevalence ($988,971/11,029 \approx 89.7$). The trained forest was exported to ONNX [17] with the skl2onnx converter 1.20.0; on the test set the ONNX scores differ from the scikit-learn probabilities by at most 2.5×10^{-7} , i.e., at float32 rounding level. This exported model is what the ONNX arms execute, so every ONNX arm and every scikit-learn arm evaluates the same trees.

The JVM-native comparison model is a Smile [19] random forest trained on the same encoded training data with nominally aligned hyperparameters (Table I). Passing the class weight through Smile's own class-weight argument produced near-constant scores in our configuration (test ROC-AUC 0.54). We therefore applied the same weight explicitly by replicating each fraud row 90 times in the training set only, which is library-independent and equivalent in expectation to weighting. Because the two libraries differ in bootstrap sampling, split search and the aggregation of tree outputs, the Smile model is a different model; it is included to represent the JVM-native library route, not as a replica of the scikit-learn forest.

TABLE I. Random Forest Hyperparameters

Setting	scikit-learn	Smile
Trees	100	100
Maximum depth	20	20
Maximum leaf nodes	500	500
Minimum leaf size	5	5
Features per split	$\sqrt{d}$	$\lfloor \sqrt{d} \rfloor = 5$
Imbalance handling	class weight 90	fraud rows $\times 90$
Random seed	42	42

C. BENCHMARK ARMS

Table II defines the eleven arms and Fig. 1 shows their structure. Arms A1–A5 score in-process: the model lives in the same process as the caller. A1 and A2 call scikit-learn with one and with all threads, respectively; A3 runs the ONNX export with ONNX Runtime in CPython; A4 runs the same ONNX file with the ONNX Runtime Java API inside the JVM; A5 runs the Smile forest. Arms A6–A11 score over HTTP on the loopback interface. A6 replicates a naive deployment: the Flask development server with request logging to the console, scikit-learn with all threads, and a new connection per request. A7–A9 use the production WSGI server waitress with scikit-learn restricted to one thread, with persistent (keep-alive) connections (A7), a new connection per request (A8) and a Java client (A9). A10 and A11 serve the ONNX model from the same waitress server to a Python and a Java client. ONNX Runtime was configured with one intra-op thread in all ONNX arms.

TABLE II. Benchmark Arms

ID	Caller → model	Engine / server
A1	Python, in-process	sklearn, n_jobs=1
A2	Python, in-process	sklearn, n_jobs=-1
A3	Python, in-process	ONNX Runtime
A4	JVM, in-process	ONNX Runtime (Java)
A5	JVM, in-process	Smile RF
A6	Python → HTTP	Flask-dev, sklearn -1, no keep-alive
A7	Python → HTTP	waitress, sklearn 1, keep-alive
A8	Python → HTTP	waitress, sklearn 1, no keep-alive
A9	Java → HTTP	waitress, sklearn 1, keep-alive
A10	Python → HTTP	waitress, ONNX, keep-alive
A11	Java → HTTP	waitress, ONNX, keep-alive

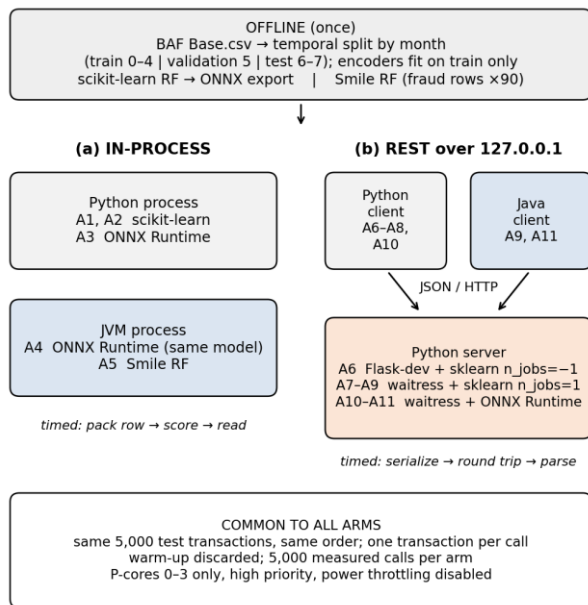


FIGURE 1. Structure of the benchmark. (a) In-process arms score inside the caller's process; (b) REST arms score over HTTP on the loopback interface. All arms replay the same 5,000 test transactions under the same controls.

D. MEASUREMENT PROTOCOL

A fixed set of 5,000 transactions, the first 5,000 rows of a seeded (seed 42) permutation of the test months, is written once to a file and replayed in the same order by every arm, one transaction per call (batch size 1), in a closed loop. Each measured call is timed individually with a monotonic high-resolution clock (`time.perf_counter` in Python, `System.nanoTime` in Java). For in-process arms the timed region covers packing the feature vector into the engine's input structure, scoring and reading the fraud probability; for REST arms it covers request serialization, the HTTP round trip, server-side parsing and scoring, and response parsing. Python arms perform 500 discarded warm-up calls; the JVM in-process arms perform 20,000 so that just-in-time compilation reaches a steady state; the Java REST client performs 500. Each arm then records 5,000 measured calls, and the complete sequence of arms was executed three times. To ensure the JVM cannot eliminate the scoring call as dead code, and to verify parity, the sum of scores over the measured calls is retained and compared across arms.

E. ENVIRONMENT AND CONTROLS

All experiments ran on one laptop with a 12th-generation Intel Core i5-1235U (two performance cores with two threads each and eight efficiency cores), 16 GB RAM, Windows 11 (build 26200), Eclipse Temurin JDK 21.0.12, Python 3.13, NumPy 2.5, pandas 3.0, scikit-learn 1.9.1 and ONNX Runtime 1.30 (Python), ONNX Runtime 1.17.1 (Java), Smile 3.1.1, Flask 3.1.3, waitress 3.0.2 and skl2onnx 1.20.0. The laptop was on mains power in the highest performance mode with background applications closed and cloud file synchronization paused.

Two controls proved essential. First, on this hybrid processor the operating system migrated benchmark

processes between performance and efficiency cores, and back-to-back runs of the same arm differed by up to a factor of three. Every process, including servers and clients, was therefore pinned to logical processors 0-3 (the two performance cores) and run at high priority. Second, Windows power throttling slows processes whose window is not in the foreground. In a diagnostic run the model-only time measured inside the scikit-learn server was 10.9-11.7 ms while the server ran in the background but 4.0 ms when its window was in the foreground, against 3.97 ms in-process. Power throttling was therefore disabled system-wide before the reported runs. The reported results come from three complete, unattended scripted runs of all arms under these controls, executed on the same day; servers and clients share the same four logical processors, as they would when co-located on one host.

F. STATISTICAL ANALYSIS

For each arm we report the median, 95th and 99th percentiles. Arms are compared with A3 (in-process ONNX Runtime in CPython) as reference. The ratio of medians is reported with a 95% confidence interval from a block bootstrap [28], [27] with a moving-block length of 20 calls and $B = 2,000$ resamples, which respects serial dependence between consecutive calls. Differences in distribution are tested with the two-sided Mann-Whitney U test [25] and their magnitude is summarized with Cliff's delta [26], which ranges from -1 to 1 and equals 1 when every observation of one arm exceeds every observation of the other. These within-run statistics were computed for each run; because the machine state differs between runs, each arm is summarized by the median of its three per-run medians and by the range across runs, and conclusions are drawn only from patterns that hold in all three runs [24]. Classification quality is reported as ROC-AUC, PR-AUC, and precision, recall and MCC at two thresholds chosen on the validation month: the threshold giving a 5% false-positive rate, as recommended for BAF [11], and the threshold maximizing F1.

IV. RESULTS

A. CLASSIFICATION PERFORMANCE

Table III reports test-set performance. The scikit-learn forest reaches ROC-AUC 0.880 and PR-AUC 0.158; the Smile forest reaches 0.851 and 0.129. At the 5% false-positive operating point the scikit-learn model detects 52.7% of fraud cases with precision 0.122 (MCC 0.234), whereas Smile detects 47.4% with precision 0.110 (MCC 0.207). At the best-F1 threshold the scikit-learn model again has the higher MCC (0.222 versus 0.205). Because the ONNX arms reproduce the scikit-learn scores to float32 precision, their classification performance is identical to that of the scikit-learn model. Absolute PR-AUC values are low, as expected for a 1.4% positive rate in the test months, and should be read against the prevalence baseline of 0.014.

TABLE III. Test-Set Classification Performance (Months 6–7)

Metric	sklearn / ONNX	Smile
ROC-AUC	0.880	0.851
PR-AUC	0.158	0.129
Precision @ 5% FPR	0.122	0.110
Recall @ 5% FPR	0.527	0.474
MCC @ 5% FPR	0.234	0.207
Precision @ best-F1	0.198	0.159
Recall @ best-F1	0.278	0.306
MCC @ best-F1	0.222	0.205

B. SCORE PARITY ACROSS ARMS

Every arm that executes the scikit-learn forest or its ONNX export produced the same scores. On the 5,000 benchmark transactions, the Java ONNX arm (A4) and the Python ONNX arm (A3) differ by at most 3.6×10^{-7} on any transaction, and the sum of scores over the measured calls is 1278.5747 in every run for A4, for the Java client against the scikit-learn server (A9) and for the Java client against the ONNX server (A11). Latency differences between these arms are therefore not accuracy differences.

C. LATENCY

Table IV and Fig. 2 summarize the three runs. Within each run, differences from the reference arm are large relative to their uncertainty: in run 1 all differences were significant ($p < 0.001$), all effect sizes satisfied $|\delta| \geq 0.66$, and the bootstrap confidence intervals of median ratios were narrow (e.g., 170–177 for A1/A3 and 1.28–1.32 for A4/A3). Variation between runs is considerably larger, particularly for the REST arms, whose medians varied by up to a factor of 2.6, whereas in-process scikit-learn varied by 10%. We therefore report ranges across runs and base conclusions only on patterns that hold in every run.

TABLE IV. Latency per Transaction over Three Runs (5,000 Calls per Arm per Run): Median and Range of Run Medians, Median p99, and Range of the Per-Run Ratio to A3

Arm	Median (μ s)	Range (μ s)	p99 (μ s)	Ratio to A3
A5	11.9	8.5–13.8	25.8	0.35–0.61
A3	24.6	22.8–33.5	59.3	1
A4	33.2	29.7–43.4	59.3	0.99–1.76
A11	852	758–975	3,422	25–40
A10	1,423	943–2,416	4,065	41–98
A1	3,972	3,667–4,032	12,097	109–174
A9	6,525	4,617–7,770	13,828	203–265
A7	7,830	4,583–8,390	15,713	201–341
A8	10,532	5,076–10,833	40,522	223–428
A2	21,515	18,440–30,615	41,973	550–1,245
A6	45,091	40,285–48,423	86,411	1,203–2,124

Three observations answer RQ1 and RQ2. First, the engine dominates. With the same trained forest, in-process scikit-learn (A1) needs a median of 3.97 ms per transaction, while ONNX Runtime executing the exported trees in the same CPython process (A3) needs 24.6 μ s; the per-run ratio

ranges from 109 to 174, consistent with earlier reports that ONNX-ML scores single records faster than scikit-learn [15]. Second, when the engine is held fixed, the host runtime matters little: the same ONNX model inside the JVM (A4) takes 33.2 μ s, between 0.99 and 1.76 times the CPython figure in the three runs, with similar p99 values (59.0–69.1 versus 51.7–83.2 μ s). The JVM is therefore never faster than CPython for the same engine; the occasional slowdown is plausibly due to per-call tensor creation in the Java API and to the older Java ONNX Runtime release. Third, the JVM-native Smile forest (A5) is the fastest arm in every run, at 8.5–13.8 μ s or 0.35–0.61 times the reference.

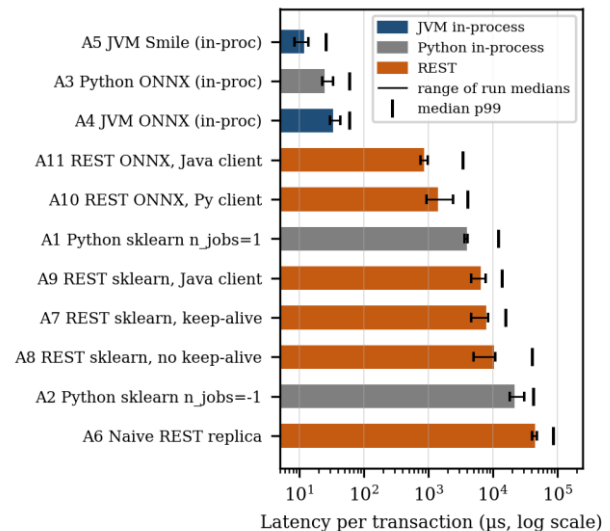


FIGURE 2. Latency per transaction over three runs, log scale. Bars show the median of the per-run medians, whiskers the range of per-run medians and ticks the median 99th percentile. Colors distinguish JVM in-process, Python in-process and REST arms.

For RQ3, the HTTP hop is small compared with the engine cost but, unlike the in-process arms, it varied between runs. The most stable measurement is the Java client against the ONNX server: moving the same ONNX model from in-process execution in the JVM (A4) to the waitress service (A11) added 0.73–0.93 ms to the median in the three runs. With the Python client (A10 versus A3) the added median was 0.92–2.39 ms, and the Java client was faster than the Python client against the same server in every run. For the scikit-learn server, the difference from in-process scikit-learn (A7 versus A1) was 0.61 ms in the first run but 4.2–4.4 ms in the second and third, more than the HTTP path adds for the ONNX model; the scikit-learn model itself therefore appears to have run more slowly inside the server in those runs. Server-side model time was not recorded in the scripted runs, so this cannot be attributed further; plausible causes are contention between server and client, which share four logical processors, and residual operating-system power management (Section III-E). Opening a new connection per request (A8) raised the median by 11–38% over A7 and the p99 by a factor of 1.6–3.4, so connection reuse is primarily a tail-latency control.

D. REVISITING THE NAIVE BASELINE

The naive deployment (A6) takes a median of 45.1 ms (40.3–48.4 ms across runs). Most of this cost is avoidable configuration rather than an inherent cost of REST serving. Restricting scikit-learn to one thread and replacing the development server by waitress with keep-alive (A7) reduces the median by a factor of 5.1–10.6 across runs, and serving the ONNX export instead (A10) by a factor of 19–51. The multi-threaded scikit-learn setting alone is costly for single rows: in-process, `n_jobs=-1` (A2) is 5.0–7.6 times slower than `n_jobs=1` (A1) and has a much heavier tail. A comparison of an in-process JVM model against A6 would therefore attribute to the JVM a speed-up that is mostly produced by the baseline.

V. DISCUSSION

A. IMPLICATIONS FOR JVM-BASED FRAUD SCORING

The results separate three decisions that are often conflated. The largest gain comes from the choice of inference engine: evaluating a 100-tree forest through a compiled runtime rather than through scikit-learn's per-call machinery reduces single-transaction latency by two orders of magnitude, in either language. The choice of host runtime is secondary once the engine is fixed. The choice between embedding and a separate service costs about 0.7–0.9 ms per transaction on one host with a Java client in our runs, more with a Python client, plus a heavier and more variable tail; this is the real, defensible advantage of embedding a model in the JVM, and it matters mainly when the latency budget is tight or when the service would otherwise have to be operated and scaled separately. Against authorization budgets measured in tens to hundreds of milliseconds, all correctly configured arms here are fast; the differences become decisive under high throughput, tight tail-latency objectives [14] or many models per decision.

The exported-model route (A4) keeps the accuracy of the Python model while running inside the JVM. The JVM-native library route (A5) is fastest but, in our experiments, less accurate, and it required a workaround for class weighting. For teams that develop in Python and deploy on the JVM, exporting to ONNX therefore offers the more favourable trade-off (RQ4). Our results are consistent with the embedded-ONNX advantage reported by Bondaruk et al. [22], but they qualify its interpretation: in our setting, the same ONNX model embedded in CPython (A3) is at least as fast as in the JVM (A4), so the advantage stems from the inference engine and from removing the HTTP hop rather than from the JVM itself.

B. METHODOLOGICAL LESSONS

Two effects that are easy to miss changed our measurements by factors of two to three. On hybrid processors, unpinned processes move between core types, so single runs are unreliable; pinning to performance cores removed most of this variation. Operating-system power management throttles background processes; because a benchmark server usually runs in the background, this inflated server-side model time roughly threefold until throttling was disabled. Neither effect is specific to Python or

to the JVM, but either could create or erase an apparent difference between them. Even with these controls, the medians of the REST arms varied by up to a factor of 2.6 between three runs on the same day, whereas in-process scikit-learn varied by 10%. We recommend that latency studies on client hardware report core pinning and power settings explicitly and repeat complete runs rather than rely on a single run.

C. THREATS TO VALIDITY

Internal validity. The JVM in-process arms warm up with 20,000 calls that cycle through the same rows that are later measured, whereas Python arms warm up with 500 calls; cache and branch-predictor state may therefore slightly favour the JVM arms. The Java and Python ONNX Runtime releases differ (1.17.1 versus 1.30), which may contribute to the A4–A3 difference. The fraud class weight was derived from the prevalence of the whole dataset rather than the training months alone; this affects only a weighting constant, not the split or the features. Construct validity. Latency is measured for single transactions in a closed loop on one host over loopback; batching, concurrency and real network latency are not covered. External validity. Results are for one model family, one synthetic dataset and one laptop; server hardware, Linux and process-based servers such as gunicorn may shift absolute numbers. Conclusion validity. The reported figures summarize three complete runs executed on the same day on one machine. The REST arms varied substantially between runs, and more runs on dedicated hardware would narrow the ranges [24]; conclusions are therefore stated only where they hold in all three runs. Because p-values are uninformative at this sample size, within-run evidence rests on effect sizes and confidence intervals.

VI. CONCLUSION AND FUTURE WORK

We benchmarked eleven ways of scoring a fraud model for a single transaction, varying the inference engine, the host runtime and the transport one at a time around the same trained random forest, with verified score parity, in three complete runs. The inference engine accounts for most of the latency: the same forest runs 109 to 174 times faster through ONNX Runtime than through scikit-learn in the same Python process. With the engine held fixed, the JVM offers no speed advantage over CPython (median 33.2 versus 24.6 μ s). A correctly configured local REST service adds roughly 0.7–0.9 ms per transaction with a Java client, more with a Python client, and a heavier tail; this is the genuine cost that in-process deployment removes, whereas a naive development-server deployment costs five to eleven times more than a tuned service. A JVM-native Smile forest is fastest but less accurate. Because REST latency on client hardware varied markedly between runs, such studies should report repeated complete runs.

Future work will repeat the benchmark on server-class Linux hardware with process-based Python servers, measure throughput and tail latency under concurrent load and batching, extend the comparison to gradient-boosted trees and neural models served through ONNX Runtime and the Deep Java Library [20], and evaluate the remaining BAF variants.

DATA AND CODE AVAILABILITY

The benchmark code, configuration, raw per-call latency files and analysis scripts are available at <https://github.com/Angad-23/JVM-fraud-benchmark> and archived on Zenodo (version 1.1, doi: 10.5281/zenodo.22998201), including the per-call files of all three runs and of the diagnostic runs described in Section III-E. The BAF dataset is available from its authors under its original license [11].

ACKNOWLEDGMENT

The author thanks the faculty of the Department of Computer Science & Engineering at Swami Vivekanand Group of Institutes and I.K. Gujral Punjab Technical University for their guidance and support. The author used Claude (Anthropic) during this work to generate and debug parts of the benchmark software, to help diagnose measurement problems, and to draft and edit the text of all sections of this manuscript. All experimental design, data collection, execution, analysis, and final decision-making were performed independently by the author, who accepts full responsibility for the contents of this paper.

REFERENCES

- [1] R. J. Bolton and D. J. Hand, "Statistical fraud detection: A review," *Statist. Sci.*, vol. 17, no. 3, pp. 235–255, Aug. 2002.
- [2] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection: A realistic modeling and a novel learning strategy," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 8, pp. 3784–3797, Aug. 2018.
- [3] D. Sculley et al., "Hidden technical debt in machine learning systems," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 28, 2015, pp. 2503–2511.
- [4] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, Oct. 2001.
- [5] S. Bhattacharyya, S. Jha, K. Tharakunnel, and J. C. Westland, "Data mining for credit card fraud: A comparative study," *Decis. Support Syst.*, vol. 50, no. 3, pp. 602–613, Feb. 2011.
- [6] F. Carcillo, Y.-A. Le Borgne, O. Caelen, Y. Kessaci, F. Oblé, and G. Bontempi, "Combining unsupervised and supervised learning in credit card fraud detection," *Inf. Sci.*, vol. 557, pp. 317–331, May 2021.
- [7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, Jun. 2002.
- [8] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009.
- [9] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, Art. no. e0118432, Mar. 2015.
- [10] D. Chicco and G. Jurman, "The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation," *BMC Genomics*, vol. 21, no. 1, Art. no. 6, Jan. 2020.
- [11] S. Jesus, J. Pombal, D. Alves, A. Cruz, P. Saleiro, R. Ribeiro, J. Gama, and P. Bizarro, "Turning the tables: Biased, imbalanced, dynamic tabular datasets for ML evaluation," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS) Datasets and Benchmarks Track*, vol. 35, 2022, pp. 33563–33575.
- [12] D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, and I. Stoica, "Clipper: A low-latency online prediction serving system," in *Proc. 14th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2017, pp. 613–627.
- [13] C. Olston et al., "TensorFlow-Serving: Flexible, high-performance ML serving," 2017, arXiv:1712.06139.
- [14] J. Dean and L. A. Barroso, "The tail at scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, Feb. 2013.
- [15] S. Nakandala, K. Saur, G.-I. Yu, K. Karanasos, C. Curino, M. Weimer, and M. Interlandi, "A tensor compiler for unified machine learning prediction serving," in *Proc. 14th USENIX Symp. Operating Syst. Design Implement. (OSDI)*, 2020, pp. 899–917.
- [16] H. Guan et al., "A comparison of end-to-end decision forest inference pipelines," in *Proc. ACM Symp. Cloud Comput. (SoCC)*, 2023, doi: 10.1145/3620678.3624656.
- [17] ONNX Runtime developers, "ONNX Runtime," 2021. [Online]. Available: <https://onnxruntime.ai>
- [18] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [19] H. Li, "Smile: Statistical machine intelligence and learning engine." [Online]. Available: <https://haifengl.github.io>
- [20] Amazon Web Services, "Deep Java Library (DJL)." [Online]. Available: <https://djl.ai>
- [21] S. Gross, "PEP 703 – Making the global interpreter lock optional in CPython," *Python Enhancement Proposals*, 2023. [Online]. Available: <https://peps.python.org/pep-0703/>
- [22] T. Bondaruk, O. Tsypliak, V. Shkaruplyo, and V. Artemchuk, "Embedded ONNX versus Python sidecar inference for anomaly-aware availability response in Spring Boot microservices: An architectural evaluation," *J. Cybersecurity Privacy*, vol. 6, no. 5, Art. no. 141, 2026, doi: 10.3390/jcp6050141.
- [23] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Program. Syst. Lang. Appl. (OOPSLA)*, 2007, pp. 57–76.
- [24] T. Kalibera and R. Jones, "Rigorous benchmarking in reasonable time," in *Proc. ACM SIGPLAN Int. Symp. Memory Manage. (ISMM)*, 2013, pp. 63–74.
- [25] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *Ann. Math. Statist.*, vol. 18, no. 1, pp. 50–60, Mar. 1947.
- [26] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions," *Psychol. Bull.*, vol. 114, no. 3, pp. 494–509, 1993.
- [27] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.
- [28] H. R. Künsch, "The jackknife and the bootstrap for general stationary observations," *Ann. Statist.*, vol. 17, no. 3, pp. 1217–1241, Sep. 1989.

BIOGRAPHY

ANGAD DHIMAN is an M.Tech. research scholar in Computer Science & Engineering at I.K. Gujral Punjab Technical University, Jalandhar, India, affiliated with the Swami Vivekanand Group of Institutes. His research interests include machine learning model serving, performance evaluation of JVM and Python systems, and real-time fraud detection.